

ReproLab: Express Control Plane and Go Execution Engine

Detailed Functional Architecture, Responsibilities, Contracts, Diagrams and Safety Rules

Abdul Moiz Hussain
ReproLab Engineering

Contents

1	Executive Boundary Between Express and Go	3
2	High-Level Architecture Diagram	3
3	Why Asynchronous Messaging Is Essential	4
4	Express Control Plane in Great Detail	4
4.1	Express Primary Responsibilities	4
4.2	Express Module Diagram	5
4.3	Express Modules and Detailed Functions	5
4.4	Express REST and WebSocket Surface	6
4.5	Express Run Creation and Live Delivery Sequence	7
4.6	Express Transactional Outbox Rule	7
4.7	Express State Guard Rules	8
4.8	Express Artifact, Share and Comparison Functions	8
4.9	Express GitHub Authentication Responsibilities	8
4.10	Express Must Not Do	9
5	Go Execution Engine in Great Detail	9
5.1	Go Primary Responsibilities	9
5.2	Go Runner Pipeline Diagram	10
5.3	Go Phase-by-Phase Responsibilities	10
5.4	Recommended Go Package Structure	11
5.5	Go Concurrency Model	11
5.6	Concurrency Is Not Capacity	12
5.7	Go Evidence Capture Responsibilities	12
5.8	Artifact Bundle Produced by Go	13
5.9	Go Must Not Do	13
6	Contracts Between Express and Go	14
6.1	Example Run Job Envelope	14
6.2	Example Normalized Event Envelope	14
6.3	Delivery and Idempotency Rules	14
6.4	Example Environment Specification	15
7	Run Lifecycle State Machine	15

8	Failure, Cancellation and Crash Recovery	16
8.1	Crash Recovery Rules	17
8.2	Required Failure Fixtures	17
9	Security Responsibilities	17
9.1	Suggested Hardened Container Policy	18
10	Deployment Boundaries	19
11	Observability and Operating ReproLab	19
11.1	Express Telemetry	19
11.2	Go Telemetry	19
11.3	Internal Correlation	19
11.4	Operational Runbooks	20
12	Testing Strategy for Express and Go	20
12.1	Golden End-to-End Fixture	20
12.2	Critical Reliability Tests	20
13	Final Summary of Express vs Go Responsibilities	21

1 Executive Boundary Between Express and Go

ReproLab is intentionally polyglot. The Express control plane and the Go execution engine have separate responsibilities and separate trust boundaries.

The most important rule is:

Express owns product state, authorization and user-facing orchestration. Go owns safe execution, evidence collection and cleanup.

Express must never start containers, execute repository commands, or hold the Docker socket. The Go runner must never make user-facing authorization decisions or store long-lived GitHub credentials.

Table 1: Primary responsibility split

Component	Owens	Must not own
Express + Type-Script	Users, sessions, GitHub App integration, projects, scenarios, run records, authorization, quotas, signed artifact access, WebSocket fan-out, run comparison metadata.	Starting containers, executing repository commands, holding the Docker socket.
Go runner	Job leases, Git checkout, Docker build/run, health checks, timeout enforcement, capture orchestration, artifact upload, heartbeats, cleanup, orphan reaping.	User-facing authorization decisions or long-lived GitHub credentials.
Playwright worker	Browser navigation, actions, assertions, screenshots, video, trace and normalized browser events.	Overall run state, container cleanup or user permissions.
PostgreSQL	Durable product metadata and runner lease state.	High-volume raw logs and videos.
Redis Streams	Durable-enough work delivery and low-latency live events.	Canonical long-term artifact storage.
S3-compatible storage	Immutable run bundles, trace files, videos, screenshots and compressed logs.	Run authorization logic.

2 High-Level Architecture Diagram

The browser never talks directly to a runner. The Express API validates the request, records a pending run in PostgreSQL and appends a versioned job to Redis Streams. A Go runner claims the job, executes it, publishes live events and uploads large artifacts directly to object storage. Express exposes authorized metadata and signed download URLs.

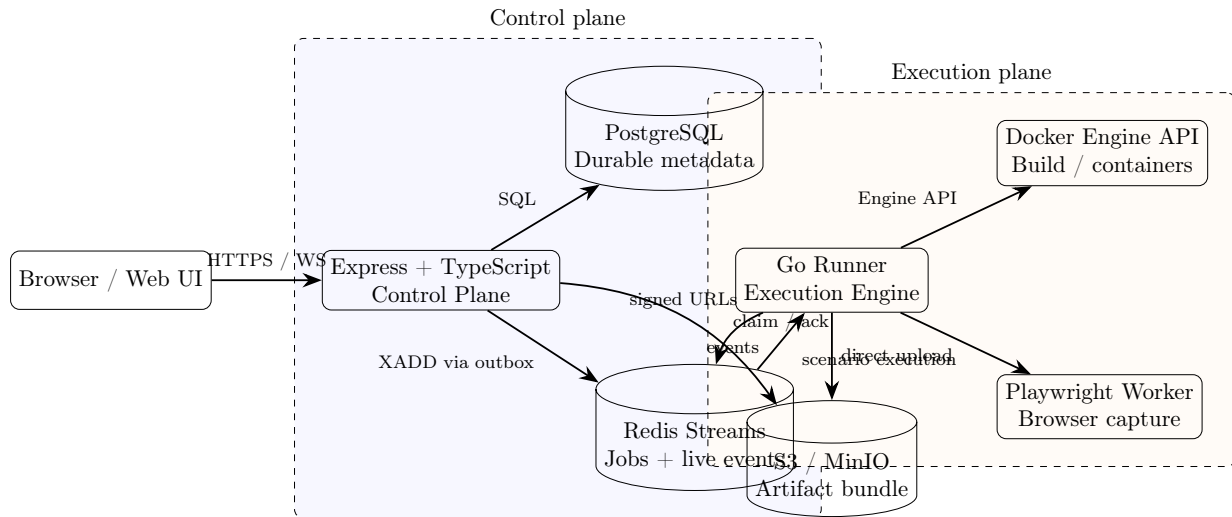


Figure 1: ReproLab control plane and execution plane

3 Why Asynchronous Messaging Is Essential

Builds and browser runs are long-running operations. They must not occupy an HTTP request life-cycle. A runner may crash, disconnect or be restarted, so every job needs a lease, acknowledgment and recovery path.

Redis Streams is used because it provides an append-only event model and consumer groups with acknowledgments. Delivery must be treated as at-least-once. Therefore:

- Every job handler must be idempotent.
- Every event handler must be idempotent.
- A runner must tolerate receiving the same job again.
- Express must deduplicate events before broadcasting or indexing.
- The canonical event stream is written as NDJSON into the artifact bundle.
- Redis may be trimmed after completion.

4 Express Control Plane in Great Detail

The Express control plane is the user-facing product brain. It authenticates users, authorizes actions, creates durable run records, publishes jobs, tracks run state, exposes artifacts and fans out live events to the browser.

4.1 Express Primary Responsibilities

Express owns:

1. User authentication and session management.
2. GitHub App installation and repository access.
3. Project creation and repository binding.
4. Scenario versioning and validation.
5. Run creation, cancellation, retry and rerun.
6. Authorization for every run, event and artifact request.
7. Quotas and abuse limits.
8. Transactional outbox publication to Redis Streams.
9. Run state transition guards using compare-and-set rules.
10. WebSocket subscription authorization and live event fan-out.

11. Artifact manifest indexing.
12. Short-lived signed object-storage URLs.
13. Run comparison metadata and environment fingerprints.
14. Runner registration views, heartbeats, capacity and draining.

4.2 Express Module Diagram

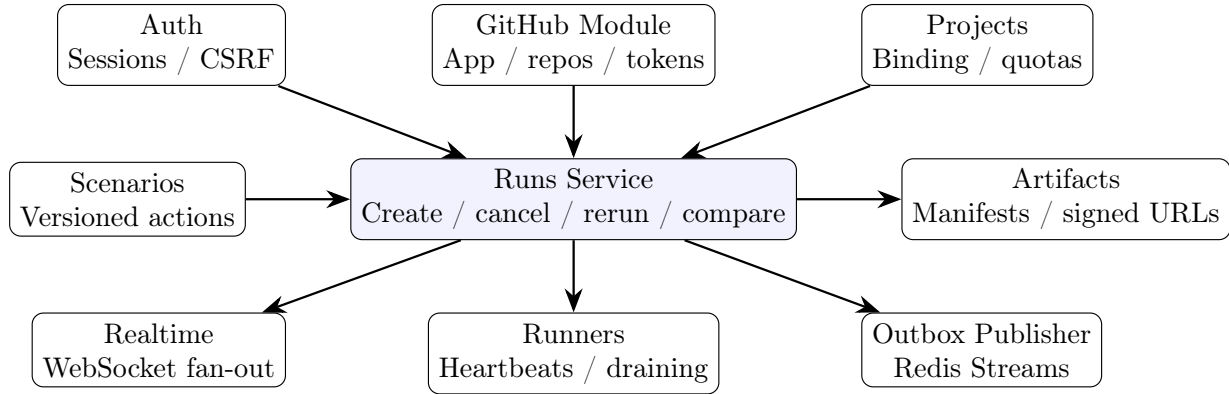


Figure 2: Express control-plane modules around the Runs service

4.3 Express Modules and Detailed Functions

Table 2: Express control-plane modules in detail

Module	Detailed functions	Constraints and rules
Auth	Session management, GitHub login, CSRF protection, account lifecycle, attaching identity to projects and runs, protecting API routes and WebSocket upgrades.	Must not execute code, start containers, or hold Docker credentials.
GitHub	GitHub App installation records, repository listing, short-lived installation tokens, webhook verification, repository permission checks, optional commit status or pull-request check links.	Use a GitHub App, not personal access tokens. Installation tokens are scoped, expire after one hour, and are minted only when needed.
Projects	Repository binding, allowed environments, team membership, default quotas, project activity views, project-level settings and ownership checks.	Must validate that the user has access to the repository before creating runs.
Scenarios	Versioned action definitions, references to committed Playwright files, scenario validation, starter templates, scenario naming and version history.	Must not accept arbitrary shell commands. Must accept only a validated, versioned ReproLab specification.
Runs	Create, authorize, cancel, retry, rerun, compare and expose run metadata. Creates run and outbox records transactionally. Guards state transitions. Persists terminal status and artifact metadata.	A run is created transactionally before a job is published. Every state transition is compare-and-set. Duplicate job delivery must not create duplicate attempts.
Artifacts	Manifest indexing, artifact metadata queries, short-lived signed download URLs, share-link authorization, retention metadata, read-only reproduction links.	Run pages use object-storage manifests rather than querying millions of log rows from PostgreSQL.

Module	Detailed functions	Constraints and rules
Realtime	Authorizes WebSocket subscriptions, fans out normalized run events, deduplicates events by (attempt_id, sequence), handles slow clients and backpressure.	A user cannot subscribe to another project's run event stream. Redis is not the permanent evidence store.
Runners	Runner registration view, heartbeat visibility, capacity reporting, administrative draining, lease expiry visibility, recovery dashboards.	Express does not directly control Docker objects. It observes runner state and coordinates recovery through durable state and events.

4.4 Express REST and WebSocket Surface

Table 3: Initial Express API surface

Route	Express responsibility
POST /v1/projects	Create a project bound to a repository.
POST /v1/projects/:projectId/scenarios	Create or version a scenario for a project.
POST /v1/projects/:projectId/runs	Authorize and create a run. Insert run and outbox transactionally.
GET /v1/runs/:runId	Return authorized run metadata, status, attempts and summary.
POST /v1/runs/:runId/cancel	Create a durable cancellation command.
POST /v1/runs/:runId/rerun	Rerun from pinned inputs and create a new comparable run.
GET /v1/runs/:runId/events?after=sequence	Return normalized events after a sequence number for polling or recovery.
GET /v1/runs/:runId/artifacts	Return authorized artifact metadata and short-lived signed URLs.
GET /v1/runs/:runId/compare/:otherRunId	Return structured comparison between two runs.
WS /v1/runs/:runId/live	Authorize and stream live normalized run events.

4.5 Express Run Creation and Live Delivery Sequence

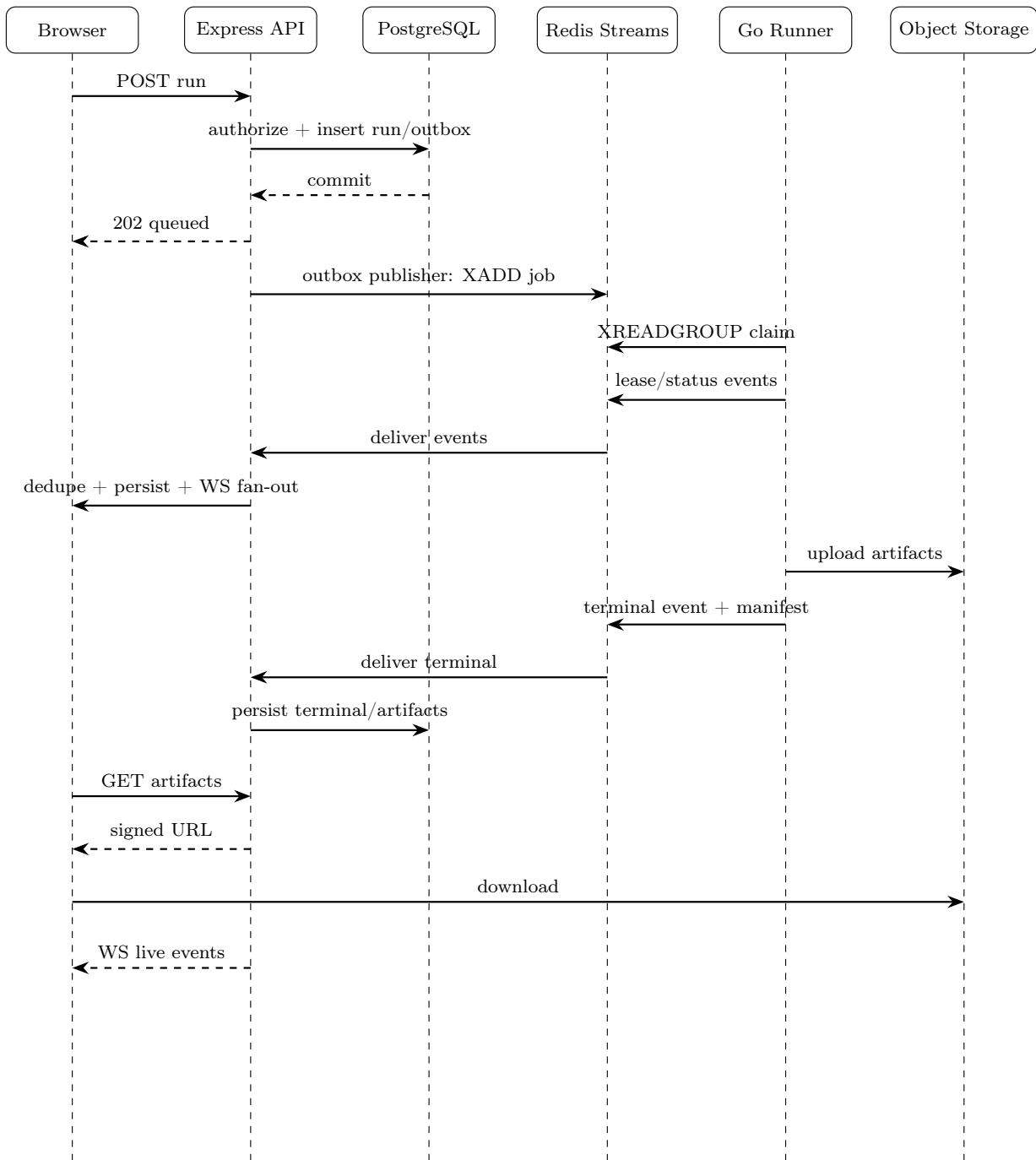


Figure 3: Express run creation, outbox publication, Go execution and artifact access

4.6 Express Transactional Outbox Rule

Express must not publish a job to Redis directly inside the HTTP request if that publication can fail after the database commit.

The correct flow is:

1. Begin a PostgreSQL transaction.
2. Insert the run record.
3. Insert an outbox event.
4. Commit the transaction.

5. A background publisher reads pending outbox events.
6. The publisher appends the job to Redis Streams.
7. The publisher marks the outbox event as published.

This guarantees that a database commit cannot succeed while job publication is lost.

4.7 Express State Guard Rules

Every state transition is compare-and-set.

Examples:

- Only a `QUEUED` run can become `LEASED`.
- Only the owning lease token can advance an attempt.
- A terminal state cannot be overwritten by a late event.
- A duplicate event with the same (`attempt_id`, `sequence`) is ignored.
- A late heartbeat cannot revive a terminated run.
- A cancellation command is durable, not only a volatile Pub/Sub message.

4.8 Express Artifact, Share and Comparison Functions

Express does not serve heavy artifact bytes directly from PostgreSQL. Instead, it uses manifests stored in object storage.

Express is responsible for:

- Indexing `manifest.json` metadata.
- Authorizing every artifact request.
- Generating short-lived signed URLs.
- Enforcing per-run object prefixes.
- Exposing read-only share links.
- Returning normalized event summaries.
- Returning comparison data between two runs.

The comparison view should include:

Table 4: Express comparison responsibilities

Dimension	Useful comparison
Scenario	First step where outcome diverged.
HTTP	Status, latency, headers and normalized body differences below a safe size threshold.
Console	New errors, removed errors and changed stack traces.
Service logs	First high-severity or correlated divergence.
Screenshots	Visual diff at matching named checkpoints.
Performance	CPU, memory and request-duration deltas.
Environment	Changed image digest, variable fingerprint or runtime version.

4.9 Express GitHub Authentication Responsibilities

Express should use a GitHub App rather than asking users for personal access tokens.

Express must:

- Store GitHub App installation records.
- Verify GitHub webhooks.
- List repositories available to an installation.
- Mint short-lived installation tokens only when needed.
- Pass credentials to the runner through an encrypted job payload or one-time secret reference.
- Avoid storing long-lived GitHub credentials.

4.10 Express Must Not Do

Express must not:

- Start containers.
- Execute repository commands.
- Hold the Docker socket.
- Query millions of raw log rows for run pages.
- Store raw GitHub installation tokens.
- Store full unredacted environment variables.
- Store videos, traces or large bundles in PostgreSQL.
- Make runner-side execution decisions.
- Trust arbitrary shell scripts from the UI.

5 Go Execution Engine in Great Detail

The Go runner is the execution engine. It claims jobs, checks out exact commits, validates the environment specification, builds and starts isolated containers, runs the Playwright scenario, collects evidence, uploads artifacts and cleans up every resource.

5.1 Go Primary Responsibilities

The Go runner owns:

1. Job leasing and acknowledgment.
2. Heartbeat reporting.
3. Isolated workspace preparation.
4. Exact Git commit checkout and verification.
5. `reprolab.yaml` validation.
6. Docker image build and pull.
7. Isolated Docker network creation.
8. Service startup and health checking.
9. Timeout enforcement.
10. Playwright worker orchestration.
11. Log, event, metric and trace collection.
12. Secret redaction before events leave the runner.
13. Artifact manifest assembly.
14. Direct artifact upload to object storage.
15. Terminal status reporting.
16. Guaranteed cleanup.
17. Orphan discovery and reaping.

5.2 Go Runner Pipeline Diagram

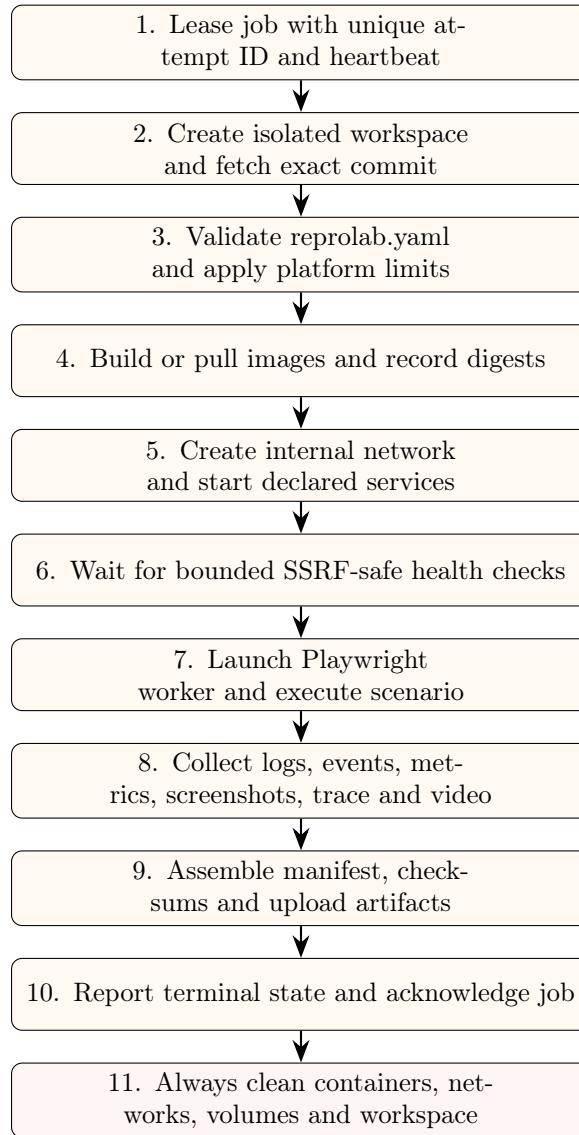


Figure 4: Go runner execution pipeline

5.3 Go Phase-by-Phase Responsibilities

Table 5: Go execution phases in detail

Phase	Detailed Go actions	Correctness and safety rules
Lease job	Claim work from Redis Streams with a unique attempt ID. Start heartbeat reporting. Stop if the lease is revoked.	Duplicate delivery must resume or exit. It must never create a second independent attempt.
Prepare workspace	Create an isolated directory. Fetch the repository. Checkout the exact commit. Verify that the resolved SHA matches the job.	Reject mismatched SHAs. Limit checkout size. Reject unsafe paths.
Validate recipe	Parse <code>reprolab.yaml</code> . Apply platform limits. Reject unsupported or dangerous options.	The runner must reject a job version it cannot understand rather than guessing.

Phase	Detailed Go actions	Correctness and safety rules
Build and pull	Use the Docker Engine API from Go to build images. Pull dependency images. Record image digests.	Every image digest or rebuild record must be captured for rerun comparison.
Network and services	Create an internal per-run Docker network. Start declared services such as app, Postgres or Redis. Apply labels for run ID, attempt ID and expiry.	Runtime egress is denied by default. Only declared internal services are reachable.
Readiness	Use bounded health checks with explicit timeouts. Wait until the application is reachable from the Playwright worker.	Health checks must be SSRF-safe. Disallow arbitrary metadata-service and host addresses.
Record scenario	Launch a Playwright worker container on the same isolated network. Execute the action DSL or committed Playwright script.	The Playwright worker does not own overall run state or cleanup.
Collect evidence	Stream logs and events. Sample container metrics. Capture browser trace, video, screenshots, console messages, network requests and WebSocket frame metadata.	Redact secret-looking values before events leave the runner.
Upload artifacts	Assemble <code>manifest.json</code> , NDJSON events, checksums and compressed logs. Upload files directly to object storage under the run/attempt prefix.	Object upload failure must leave a visible partial-artifact error, not false success.
Terminal report	Publish terminal status and artifact manifest. Acknowledge the job only after terminal status and manifest are persisted.	A job is acknowledged only after durable evidence exists.
Cleanup	Stop and remove containers, networks, temporary volumes and workspaces even after cancellation or failure.	Cleanup must run on success, failure, timeout, cancellation and crash recovery.
Recovery	Detect expired leases. Mark attempts abandoned. Append failure events. Return jobs to the queue only when retry policy permits.	The reaper removes expired labeled Docker objects independently of the original goroutine.

5.4 Recommended Go Package Structure

Listing 1: Recommended Go runner package layout

```

services/runner/
  cmd/runner/           # process entrypoint and configuration
  internal/jobs/         # Redis Streams consumer and lease handling
  internal/contracts/    # versioned job and event types
  internal/gitx/         # clone, checkout and commit verification
  internal/spec/         # reprotlab.yaml validation
  internal/executor/     # run state machine
  internal/dockerx/      # Engine API adapter
  internal/capture/      # logs, metrics and Playwright orchestration
  internal/artifacts/    # bundle manifest, compression and upload
  internal/security/     # limits, allowlists, redaction and policy
  internal/reaper/       # orphan discovery and cleanup
  internal/telemetry/    # runner traces, metrics and structured logs

```

5.5 Go Concurrency Model

Use one goroutine per active run plus dedicated goroutines for:

- Job consumption.
- Command consumption, including cancellation.
- Heartbeat reporting.
- Orphan reaping.

Inside a run, use an errgroup-style cancellation tree. If cloning, building, startup or recording fails, cancel sibling collectors and move immediately to collection and cleanup.

Bound total concurrency with a weighted semaphore based on runner capacity.

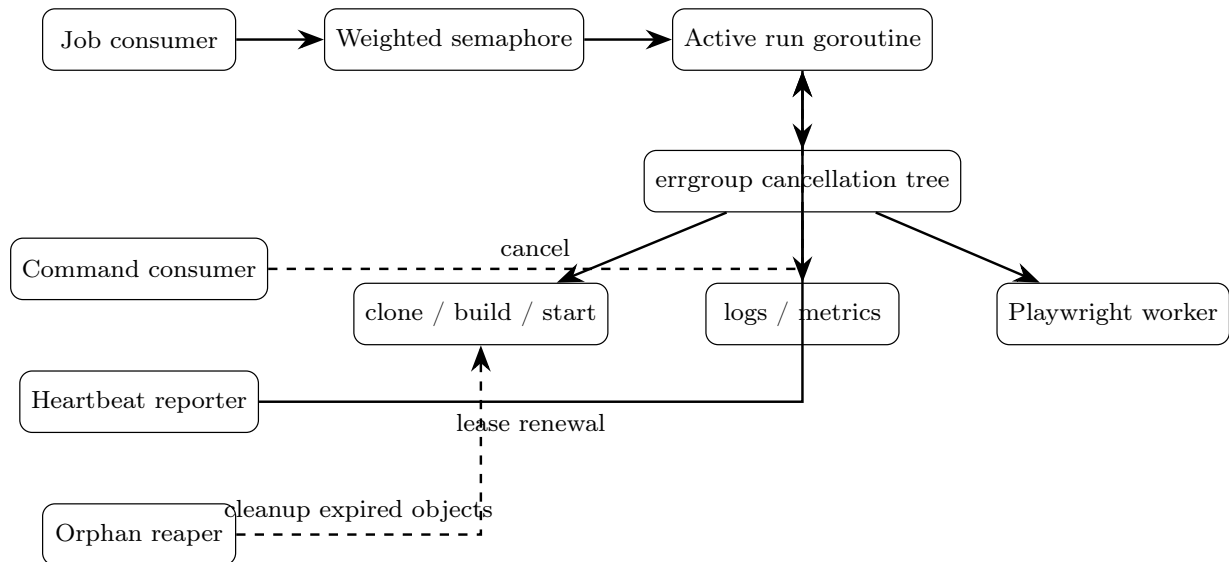


Figure 5: Go runner concurrency and cancellation model

5.6 Concurrency Is Not Capacity

A Go runner can manage many concurrent streams, but Docker builds and browsers consume real CPU, memory and disk.

Therefore:

- Start with one active run per runner host.
- Raise capacity only after collecting resource measurements.
- Do not use unbounded goroutines or channels.
- Every phase must have an explicit timeout.
- Use a weighted semaphore to bound active runs.

5.7 Go Evidence Capture Responsibilities

The Go runner orchestrates evidence capture even though the Playwright worker produces browser evidence.

Browser evidence includes:

- Playwright trace archive.
- Video recording, preferably retained on failure for cost control.
- Screenshots at named checkpoints and automatically on failure.
- Normalized action events: goto, click, fill, assertion start, assertion result, navigation.
- Console messages and uncaught page errors.
- Request/response metadata.
- WebSocket frame metadata.

Service evidence includes:

- Timestamped stdout and stderr from every selected container.
- Container start, stop, exit code, OOM status and health-check results.
- CPU, memory, process and network samples at a modest interval.
- Optional OpenTelemetry spans from applications that already emit OTLP.

Unified timeline rules:

- Every event receives an attempt-local sequence number at ingestion.
- Store both event time and ingestion time.
- Use correlation IDs when available: trace ID, span ID, HTTP request ID, application IDs.
- Group events into lanes: browser, network, app, database, infrastructure and annotations.
- Redact secret-looking values before events leave the runner.
- Perform a second redaction pass before final bundle publication.

5.8 Artifact Bundle Produced by Go

Listing 2: Canonical run bundle structure

```
run-bundle/
  manifest.json
  source.json
  environment.json
  events.ndjson.gz
  checksums.sha256

  browser/
    trace.zip
    video.webm
    screenshots/

  network.har

  services/
    app.stdout.ndjson.gz
    app.stderr.ndjson.gz
    database.stdout.ndjson.gz

  metrics/
    containers.ndjson.gz

  annotations/
    comments.json
```

5.9 Go Must Not Do

The Go runner must not:

- Make user-facing authorization decisions.
- Store long-lived GitHub credentials.
- Accept arbitrary shell commands from the control plane.
- Guess the meaning of an unknown job version.
- Acknowledge a job before terminal status and artifact manifest are persisted.
- Leave containers, networks, volumes or workspaces behind after failure.
- Allow the API or sandbox containers to access the Docker socket.
- Run privileged containers in the hardened demo.
- Permit unrestricted runtime egress by default.
- Publish unredacted secrets into events or bundles.

6 Contracts Between Express and Go

The polyglot boundary must be explicit. Place JSON Schema or Protocol Buffer definitions in a shared contracts directory, version every message and generate TypeScript and Go types when practical.

6.1 Example Run Job Envelope

Listing 3: Example run job sent from Express to Go

```
{
  "version": 1,
  "jobId": "job_01...",
  "runId": "run_01...",
  "attemptId": "attempt_01...",
  "repository": {
    "cloneUrl": "https://github.com/org/repo.git",
    "commitSha": "40-character-sha",
    "credentialRef": "secret://github/install-token/one-time-id"
  },
  "specObjectKey": "runs/run_01/spec.json",
  "limits": {
    "wallTimeSeconds": 900,
    "memoryBytes": 2147483648,
    "cpuCount": 2,
    "pids": 256,
    "diskBytes": 10737418240
  },
  "artifactUploadPrefix": "runs/run_01/attempt_01/"
}
```

6.2 Example Normalized Event Envelope

Listing 4: Example normalized event emitted by Go

```
{
  "version": 1,
  "runId": "run_01...",
  "attemptId": "attempt_01...",
  "sequence": 184,
  "occurredAt": "2026-07-16T17:42:03.481Z",
  "source": "container:app",
  "type": "service.log",
  "payload": {
    "stream": "stderr",
    "message": "checkout failed: inventory reservation timeout"
  }
}
```

6.3 Delivery and Idempotency Rules

- A job is acknowledged only after its terminal status and artifact manifest are persisted.
- The unique key is (`run_id`, `attempt_id`).
- A duplicate delivery resumes or exits; it never creates a second independent attempt silently.
- Events have monotonically increasing runner-assigned sequence numbers within an attempt.
- Express deduplicates events by (`attempt_id`, `sequence`) before broadcasting or indexing.

- The canonical event stream is written as NDJSON into the artifact bundle.
- Redis may be trimmed after completion.
- Cancellation is a durable command, not only a volatile Pub/Sub message.

6.4 Example Environment Specification

Listing 5: Example reprolab.yaml validated by Go and managed by Express

```
version: 1

source:
  context: .
  dockerfile: Dockerfile

services:
  app:
    build: source
    internalPort: 3000
    health:
      path: /health
      timeoutSeconds: 60
    environment:
      NODE_ENV: test
      DATABASE_URL: secret://generated/postgres-url

  database:
    image: postgres:17-alpine
    environment:
      POSTGRES_DB: app
      POSTGRES_USER: app
      POSTGRES_PASSWORD: secret://generated/db-password

scenario:
  type: actions
  file: .reprolab/scenarios/checkout.yaml
  browser: chromium

capture:
  browserTrace: true
  video: true
  screenshots: on-failure
  console: true
  network: true
  websocketFrames: true
  serviceLogs: [app]

limits:
  wallTimeSeconds: 600
  memoryMiB: 2048
  cpus: 2
  pids: 256
  runtimeEgress: deny
```

7 Run Lifecycle State Machine

The run lifecycle is guarded by Express and advanced by Go through events.

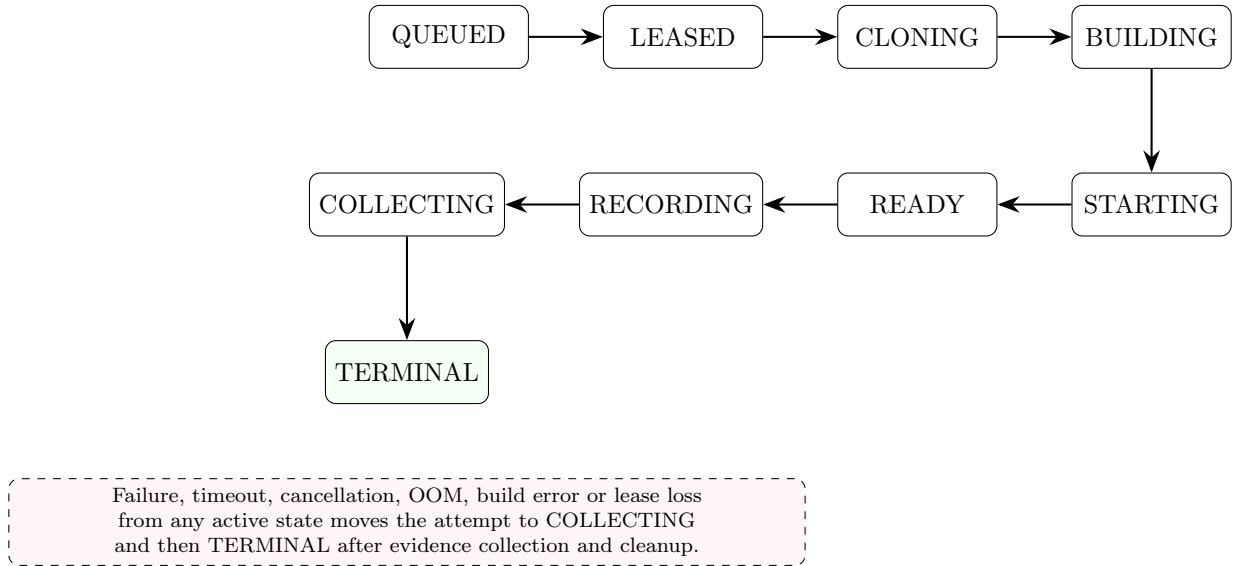


Figure 6: ReproLab run lifecycle

Table 6: State entry and exit conditions

State	Entry condition	Exit requirement
QUEUED	Run record and outbox item committed.	Runner claims the job and creates an attempt lease.
LEASED	Valid runner and lease token assigned.	Workspace created and exact commit resolved.
CLONING BUILDING	Repository credentials available. Spec validated.	Checkout SHA verified and spec loaded. All required images built or pulled with digests recorded.
STARTING READY	Network and service definitions ready. Application reachable from the Playwright worker.	Required health checks pass. Capture collectors started.
RECORDING	Browser scenario running.	Scenario passes, fails, times out or is canceled.
COLLECTING	Execution stopped.	Manifest and required artifacts uploaded.
TERMINAL	Success, failure or cancellation persisted.	Cleanup completes or is delegated to the reaper.

8 Failure, Cancellation and Crash Recovery

ReproLab must remain safe when runners crash, builds fail, containers OOM, health checks time out or users cancel runs.

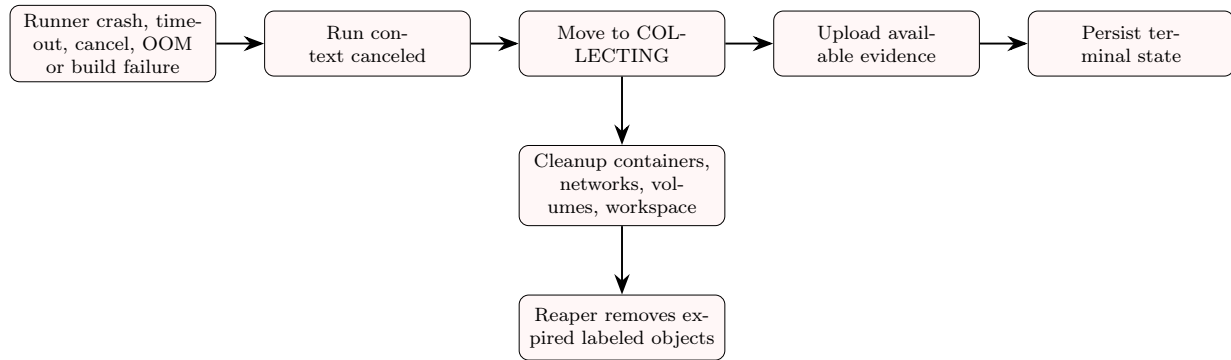


Figure 7: Failure, cancellation and recovery flow

8.1 Crash Recovery Rules

- Each runner writes a heartbeat with its capacity and active attempt IDs.
- A lease expires if its heartbeat is not renewed within a short grace period.
- A recovery worker marks the attempt abandoned.
- The recovery worker appends a failure event.
- The job returns to the queue only when retry policy permits.
- Every Docker object receives labels for run ID, attempt ID and expiry time.
- The reaper scans for expired labeled containers, networks and volumes.
- The reaper removes expired objects independently of the original goroutine.

8.2 Required Failure Fixtures

The runner should be tested against at least these failure fixtures:

1. Clone failure.
2. Build failure.
3. Failed health check.
4. Timeout.
5. OOM.
6. Cancellation during build.
7. Cancellation during startup.
8. Cancellation during recording.
9. Object upload failure.
10. Runner process killed mid-run.

9 Security Responsibilities

A normal Docker container is not a sufficient trust boundary for openly executing arbitrary hostile repositories. The first version should use trusted repositories only. Hardened execution uses a dedicated runner host, rootless Docker, gVisor and strict limits.

Table 7: Express and Go security responsibilities

Risk	Express responsibility	Go responsibility
Host takeover	Never hold Docker socket. Never execute repository commands.	Use dedicated runner VM, rootless Docker, gVisor, non-root user, no privileged containers.
Resource exhaustion	Enforce quotas and abuse limits.	Enforce hard memory, CPU, PID, disk and wall-time limits.
Docker socket abuse	Do not expose Docker to API or sandbox containers.	Only the Go runner may reach the Docker Engine API.
Network abuse	Authorize runs and artifact access.	Separate build and runtime networks. Deny runtime egress by default.
Secret theft	Mint short-lived GitHub tokens only when needed. Use secret references.	Redact secrets before events leave the runner and before bundle publication.
Data leakage	Authorize every artifact request. Issue short-lived signed URLs.	Upload to per-run object prefixes. Checksum and compress artifacts.
Orphaned resources	Show lease expiry and runner health.	Label Docker objects and run a reaper process.
SSRF	Validate declared health-check destinations in specs.	Enforce SSRF-safe health checks only against declared internal services.
Malicious archives	Reject oversized scenario or project uploads.	Limit checkout size, artifact count and compression ratios. Reject unsafe paths.

9.1 Suggested Hardened Container Policy

Listing 6: Suggested hardened runtime policy

```
runtime: runsc                                # gVisor in the hardened demo
user: non-root
readOnlyRootFilesystem: true
capDrop: [ALL]
noNewPrivileges: true
memory: 2GiB
cpus: 2
pidsLimit: 256
runtimeNetwork: internal-only
timeout: 10m
hostMounts: none
privileged: false
```

10 Deployment Boundaries

Table 8: Recommended deployment boundaries

Environment	Control plane	Execution plane	Notes
Local development	Next.js, Express, Postgres, Redis and MinIO through Docker Compose.	Run the Go runner as a host process connected to a local rootless Docker Engine.	Avoid mounting a privileged Docker socket into the API container.
Portfolio deployment	Small API host or container service plus managed Postgres, Redis and object storage.	Dedicated Linux VM for the Go runner and sandbox runtime.	Do not place untrusted builds on the API host.
Later scale	Multiple stateless API replicas.	Runner pool grouped by capacity and runtime version.	Use leases, heartbeats and scheduling constraints before considering Kubernetes.

11 Observability and Operating ReproLab

Express and Go must both emit telemetry for the platform itself. Instrument Express and Go with OpenTelemetry before attempting to instrument every user application.

11.1 Express Telemetry

Express should track:

- Run creation rate and queue depth.
- WebSocket subscribers.
- Dropped or backpressured live events.
- Redis pending entries and oldest unacknowledged job age.
- Artifact signed URL requests.
- Authorization failures.
- Quota violations.
- Outbox publication lag.

11.2 Go Telemetry

The Go runner should track:

- Run duration by phase: queue, clone, build, startup, record, collect and cleanup.
- Runner capacity and active attempts.
- Lease expirations.
- Orphan removals.
- Artifact upload bytes and failures.
- Retention deletion counts.
- Blocked egress attempts.
- Rejected specs.
- Redaction matches.
- Docker object cleanup counts.

11.3 Internal Correlation

Create one correlation context from the API request through the Redis job, Go runner logs and artifact manifest. This allows debugging of the platform itself.

11.4 Operational Runbooks

Important operational procedures include:

- Drain a runner without accepting new jobs.
- Recover a stuck pending Redis entry.
- Reconcile active attempts after a runner restart.
- Remove leaked resources by label and expiry.
- Rotate GitHub App keys and object-storage credentials.
- Disable public run creation during suspected abuse.

12 Testing Strategy for Express and Go

Table 9: Testing responsibilities by layer

Layer	Express tests	Go tests
Unit	Authorization, state transition guards, outbox creation, signed URL policy, GitHub webhook validation.	Spec validation, lease behavior, cleanup stack, event sequencing, redaction, artifact manifest checksums.
Contract	Validate fixture messages against generated TypeScript types.	Validate the same fixture messages against generated Go types.
Integration	Real Redis Streams, Postgres and MinIO. Test run creation and event deduplication.	Docker Engine adapter tested with fixture containers. Test build, startup and cleanup.
End-to-end	Launch broken-shop, execute checkout scenario, assert timeline evidence and artifacts.	Execute full runner pipeline from job claim to artifact upload and cleanup.
Failure injection	Cancel run, expire lease, simulate duplicate events, simulate slow Web-Socket clients.	Kill runner, OOM container, fail health check, fill disk quota, interrupt upload.
Security	Ensure users cannot access other projects' runs or artifacts.	Ensure secret fixtures never appear in UI events, bundles or logs.

12.1 Golden End-to-End Fixture

Create a tiny application with a repeatable checkout bug:

- The frontend sends `POST /api/orders`.
- The backend emits a request ID.
- The backend fails when a seeded inventory row is absent.
- The expected ReproLab result includes the browser click.
- The result includes the 500 response.
- The result includes the backend log containing the same request ID.
- The result includes a failure screenshot.
- The result includes a non-zero scenario result.

12.2 Critical Reliability Tests

- Running the same job message twice does not create two uncontrolled attempts.
- A canceled run reaches a terminal state and removes all resources.
- A runner crash leaves resources that the reaper removes after expiry.
- Object upload failure leaves a visible partial-artifact error rather than reporting success.
- A user cannot subscribe to another project's run event stream.
- A secret included in application output is redacted before storage.

13 Final Summary of Express vs Go Responsibilities

Table 10: Final boundary summary

Concern	Express	Go
User trust	Authenticates and authorizes users.	Trusts only validated jobs from Express.
Run creation	Creates run and outbox transactionally.	Claims published jobs through Redis Streams.
Execution	Does not execute code.	Executes builds, containers and browser scenarios.
Docker	Must not hold Docker socket.	Owns Docker Engine API access.
GitHub credentials	Mints short-lived tokens when needed.	Uses one-time secret references and does not persist long-lived tokens.
State	Guards durable state transitions.	Reports state events with monotonic sequences.
Evidence	Indexes manifests and authorizes access.	Collects, compresses, checksums and uploads evidence.
Failure	Persists terminal failure and exposes recovery status.	Cleans up resources and supports reaping.
Security	Enforces authorization, quotas and signed URLs.	Enforces isolation, limits, redaction and cleanup.

Express makes the run legitimate. Go makes the run real, observable and safe.